Attendance Code: 027021

CMP2028M

Natural Language Processing

Week 1: “Tokenization and Regular Expressions”

Heriberto Cuayahuitl

https://staff.Lincoln.ac.uk/hcuayahuitl

Delivery Team

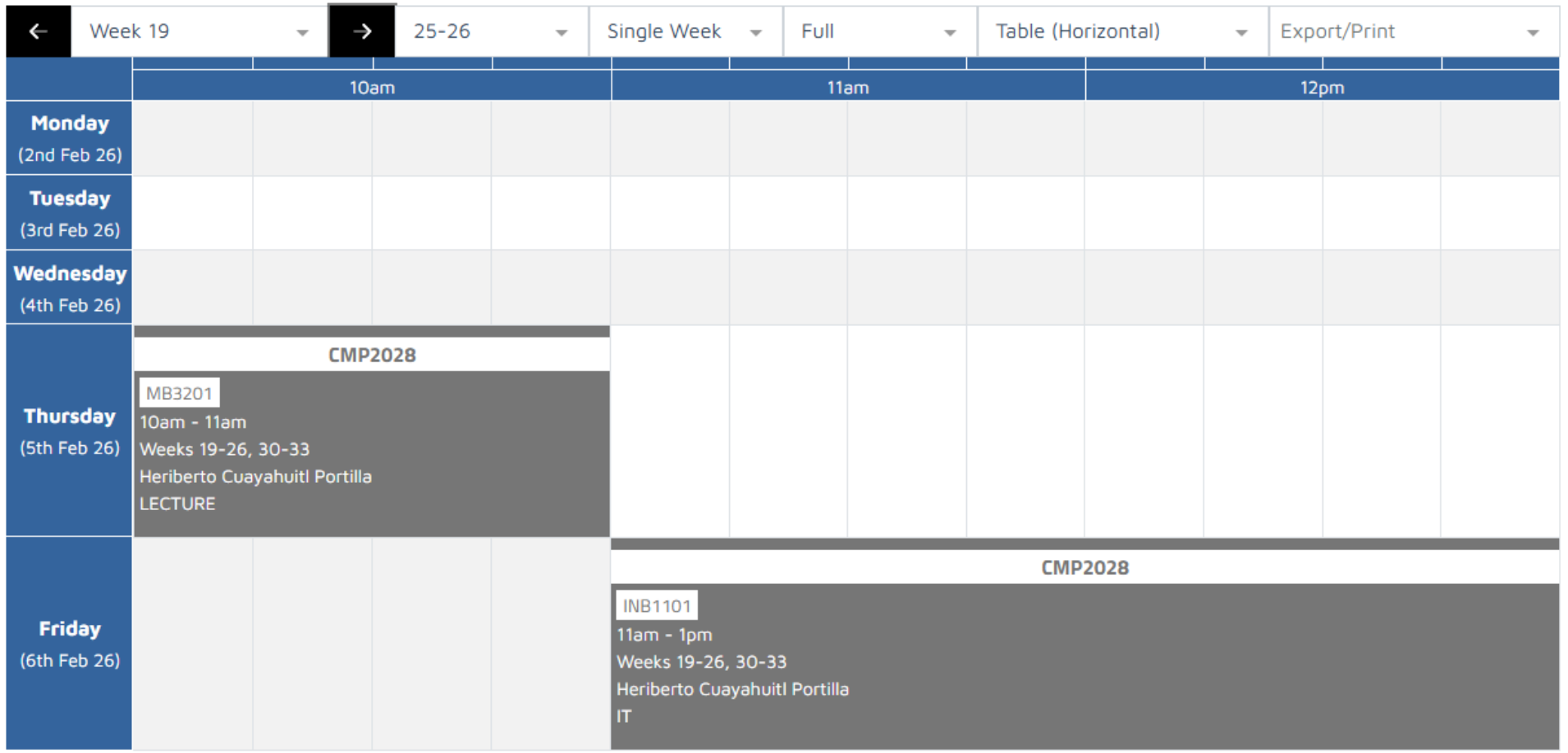
Dr. Heriberto Cuayahuitl
hcuayahuitl@lincoln.ac.uk
Office: INB3107
Surgery hours: Thursday 11hrs
Phone: 07902647611 (for emergencies)



- British computer scientist (born in Mexico)
- PhD from the University of Edinburgh in 2009
- Areas of interest:
  - Conversational AI
  - Robot Learning

Lectures and Workshops

- Lectures: Thursday at 10am in NDH0020
- Workshops: Friday 11am-1pm in INB1101



Learning Objectives

- [LO1] Demonstrate an understanding of the fundamental principles of NLP.
- [LO2] Implement a complete NLP processing pipeline using industry-standard tools.
- [LO3] Design and develop a software solution to a real-world NLP problem.
- [L04] Demonstrate a working knowledge of state-of the-art NLP research.

Agenda

- Week 1, 5-6 Feb: Tokenization and regular expressions
- Week 2, 12-13 Feb: N-gram language models
- Week 3, 19-20 Feb: Text classification
- Week 4, 27-27 Feb: Vector semantics and embeddings
- Week 5, 5-6 Mar: Large Language Models (LLMs)
- Week 6, 12-13 Mar: Question answering & information retrieval
- Week 7, 19-20 Mar: Constituent and dependency parsing
- Week 8, 26-27 Mar: Information extraction
- Week 9, 23-24 Apr: Overview of neural networks
- Week 10, 30 Apr-1May: Machine translation
- Week 11, 7-8 May: Chatbots and dialogue systems
- Week 12, 14-15 May: Speech recognition and synthesis

Easter break

Assessments



- **Assignment** (50%):
  - Multi-Choice Question Answering
  - Natural Language Processing Techniques

- **In-class test** (50%):
  - Mock-test (last workshop)
  - Test (see Hand-in deadlines and your timetable)

- **READ** the assessment documents in Blackboard.

Today

- **Preliminaries of words**
- Tokenization
- Regular expressions

Acknowledgement: Images from https://web.stanford.edu/~jurafsky/slp3/2.pdf

How Many Words in a Sentence?

- **Example sentence and utterance:**

❑ *They picnicked by the pool, then lay back on the grass and looked at the stars.*

❑ *I do uh main- mainly business data processing*

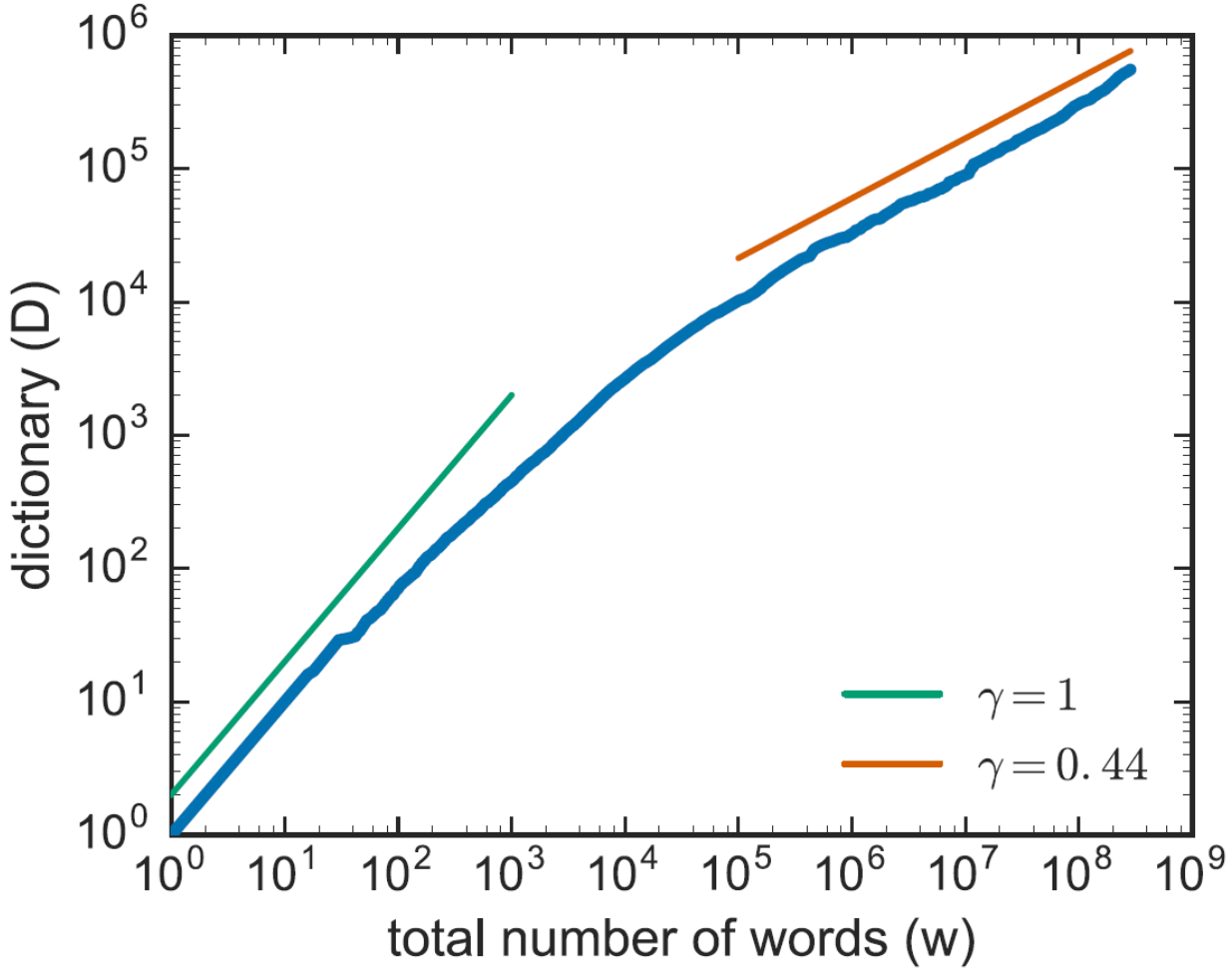
- **Word types and instances:**

  V =Unique set of words (vocab), $|V|$ =Vocabulary size

  N =total number of words in a sentence/doc/corpus/…

- **Considerations for creating a vocabulary**:
  - Do we consider capital characters or not?
  - Do we consider punctuation or not?
  - Are contractions a single word or multiple ones?

Word Types and Instances



The thick blue line shows vocabulary (V=D) as a function of text length (N=w) according to $|V| = kN^{\gamma}$.

Tria et al. “Zipf’s, heaps’ and taylor’s laws are determined by the expansion into the adjacent possible”. Entropy, 2018.

Parts of Words: Morphemes

- A **morpheme** is minimal meaning-bearing unit in a language, and **morphology**=the study of morphemes.

- Examples:
  - `Fox: consists of one morpheme (fox)`
  - `Cats: consists of two morphemes (cat, -s)`

- Classes of morphemes:
  - Roots: The central word supplying the meaning
  - Affixes: They add additional meaning
  - Example 1: `worked` => root=`work`, affix=`ed`
  - Example 2: `glasses` => root=`glass`, affix=`es`

Unicode

The **Unicode** standard is a method for representing text written using any character in any language.



U+0061	a	LATIN SMALL LETTER A
U+0062	b	LATIN SMALL LETTER B
U+0063	c	LATIN SMALL LETTER C
U+00F9	ù	LATIN SMALL LETTER U WITH GRAVE
U+00FA	ú	LATIN SMALL LETTER U WITH ACUTE
U+00FB	û	LATIN SMALL LETTER U WITH CIRCUMFLEX
U+00FC	ü	LATIN SMALL LETTER U WITH DIAERESIS
U+8FDB	进	
U+8FDC	远	
U+8FDD	违	
U+8FDE	连	
U+1F600	😀	GRINNING FACE
U+1F00E	🀎	MAHJONG TILE EIGHT OF CHARACTERS

It assigns a unique code point to each character.

UTF-8 (Unicode Transformation Format 8) represents characters efficiently and it is widely supported by tools.

Today

- Preliminaries of words
- **Tokenization**
- Regular expressions

Acknowledgement: Images from https://web.stanford.edu/~jurafsky/slp3/2.pdf

Subword Tokenization

- **Tokenization** is the first stage of natural language processing, is the process of segmenting the running input text into tokens—for standardization purposes.

- Words, morphemes and characters are three candidates for tokens. But they all have limitations.
  - ❑ Words and morphemes seem approximately at the right level for NLP processing, but they are challenging to define and to work with in practice.
  - ❑ Characters are clearer to define, but are too small as a unit for tokens.

Tokenization Algorithms

Two widely used algorithms:

- Byte-Pair Encoding (BPE)†
- Unigram Language Modeling (ULM)‡

Stages in BPE:

1. **Training**: Iteratively merges frequent neigbouring tokens to create longer and longer tokens.
2. **Encoder**: It is used to tokenize a test sentence.

† Sennrich, et al. "Neural machine translation of rare words with subword units". ACL, 2016.

‡ Kudo, et al. "Subword regularization: Improving neural network translation models with multiple subword candidates". ACL, 2018.

Byte-Pair-Encoding (BPE): Example

Tiny corpus: `set new new renew reset renew`

First, list words and vocabulary (V):

corpus		vocabulary
2	␣ n e w	␣, e, n, r, s, t, w
2	␣ r e n e w	
1	s e t	
1	␣ r e s e t	

Next, merge most frequent symbols and update V:

corpus		vocabulary
2	␣ ne w	␣, e, n, r, s, t, w, ne
2	␣ r e ne w	
1	s e t	
1	␣ r e s e t	

Byte-Pair-Encoding (BPE): Example

Repeat, merge most frequent symbols and update V:

corpus		vocabulary
2	␣ new	␣, e, n, r, s, t, w, ne, new
2	␣ r e new	
1	s e t	
1	␣ r e s e t	

Again:

corpus		vocabulary
2	␣ new	␣, e, n, r, s, t, w, ne, new, ␣r, ␣re
2	␣re new	
1	s e t	
1	␣re s e t	

BPE: Example and Algorithm

And carrying on with a few more merges we get:

merge	current vocabulary
(␣, new)	␣, e, n, r, s, t, w, ne, new, ␣r, ␣re, ␣new
(␣re, new)	␣, e, n, r, s, t, w, ne, new, ␣r, ␣re, ␣new, ␣renew
(s, e)	␣, e, n, r, s, t, w, ne, new, ␣r, ␣re, ␣new, ␣renew, se
(se, t)	␣, e, n, r, s, t, w, ne, new, ␣r, ␣re, ␣new, ␣renew, se, set

Algorithm:

function BYTE-PAIR ENCODING(strings C, number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters
for $i = 1$ **to** k **do** # merge tokens k times
  $t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C
  $t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating
  $V \leftarrow V + t_{NEW}$ # update the vocabulary
  Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus
return V

Today

- Preliminaries of words
- Tokenization
- **Regular expressions**

Acknowledgement: Images from https://web.stanford.edu/~jurafsky/slp3/2.pdf

Regular Expressions (regex)

Regular expressions are very useful for text processing.

Two widely used tools:

- Command `grep` in Unix/Linux terminals
- In Python: `re.search(pattern,string)`

  or `re.findall(pattern,string)`

The square bracket:

Pattern	Match	String
r"[mM]ary"	Mary or mary	"Mary Ann stopped by Mona's"
r"[abc]"	'a', 'b', *or* 'c'	"In uomini, in soldati"
r"[1234567890]"	any one digit	"plenty of 7 to 5"

Unix/Linux example: `echo "Mary Ann stopped by Mona's" | grep -o "[mM]ary"`

Regex: Patterns with Square Brackets

Patterns with letters and digits:

Regex	Match	Example Patterns Matched
`r"[A-Z]"`	an upper case letter	"we should call it 'Drenched Blossoms' "
`r"[a-z]"`	a lower case letter	"my beans were impatient to be hoed!"
`r"[0-9]"`	a single digit	"Chapter 1: Down the Rabbit Hole"

The caret character ^:

Regex	Match (single characters)	Example Patterns Matched
`r"[^A-Z]"`	not an upper case letter	"Oyfn pripetchik"
`r"[^Ss]"`	neither 'S' nor 's'	"I have no exquisite reason for't"
`r"[^.]"`	not a period	"our resident Djinn"
`r"[e^]"`	either 'e' or '^'	"look up ^ now"
`r"a^b"`	the pattern 'a^b'	"look up a^ b now"

Example for a string of digits: `r"[0-9][0-9]*" or r"[0-9]+"`

Regex: Counting and Anchors

Counting:

Regex	Match
*	zero or more occurrences of the previous char or expression
+	one or more occurrences of the previous char or expression
?	zero or one occurrence of the previous char or expression
{n}	exactly *n* occurrences of the previous char or expression
.	any single char
.*	any string of zero or more chars

Anchors:

Regex	Match
^	start of line
$	end of line
\b	word boundary
\B	non-word boundary

Regex: Precedence and Disjuction

Precedence, from highest to lowest:

Parenthesis	()
Counters	* + ? {}
Sequences and anchors	the ^my end$
Disjunction	|

Examples:

Regex	Description
`r"the"`	Find cases of article ***the***
`r"[tT]he"`	Same as above but with lower or uppercase ***t***
`r"\b[tT]he\b"`	Same as above but with word boundaries
`r"\b([tT]he) +(cat|dog)\b"`	Find cases of ***the cat, the dog, The cat*** *or* ***The dog***

Regex: More Operators

Aliases:

Regex	Expansion	Match	First Matches
\d	[0-9]	any digit	Party␣of␣5
\D	[^0-9]	any non-digit	Blue␣moon
\w	[a-zA-Z0-9_]	any alphanumeric/underscore	Daiyu
\W	[^\w]	a non-alphanumeric	!!!!
\s	[␣\r\t\n\f]	whitespace (space, tab)	in_Concord
\S	[^\s]	Non-whitespace	in␣Concord

Escaped characters:

Regex	Match	First Patterns Matched
*	an asterisk "*"	"K*A*P*L*A*N"
\.	a period "."	"Dr. Livingston, I presume"
\?	a question mark	"Why don't they come and lend a hand?"
\n	a newline	
\t	a tab	

Regexp: Substitutions and Capture Groups

Substitutions:

- Command `re.sub(pattern, repl, string)`
- Examples:
  - `re.sub(r"cherry", r"apricot", string)`
  - `re.sub(r"janet", r"Janet", string)`

Substitues cherry for apricot in string.

Capture group:

- Command `re.sub(r"(\d{2})/(\d{2})/(\d{4})", r"\2-\1-\3", string)}`
- Changes a date format, e.g., 10/15/2011 to 15-10-2011

Regex: Pre-tokenization

```
>>> import regex as re
>>> pat = re.compile(
... # Contractions: 't and 'm are tokens
...     r"'s|'t|'re|'ve|'m|'ll|'d|"
... # Words:  sequence of Unicode letters (after optional space)
...     r" ?\p{L}+|"
... #Number: sequence of digits (after optional space)
...     r" ?\p{N}+|"
... # Punctuation: sequence of non-alphanumeric/non-space
...                         #(after optional space)
...     r" ?[^\s\p{L}\p{N}]+|"
... # whitespace
...     r"\s+(?!\S)|\s+"
... )
>>> text = "We're 350 dogs! Um, lunch?"
>>> print(pat.findall(text))
['We', "'re", ' 350', ' dogs', '!', ' Um', ',', ' lunch', '?']
>>>
```

The GPT-2 pre-tokenizer regular expression used to split sentences before running BPE, see Fig. 2.15 of SLP book.

Today

- Introduction to module: learning objectives, assessments, timetable, recommendations.
- ***Tokenization*** is a key stage for processing raw text into units called *tokens*. Types of tokens: words, morphemes, characters, and subwords.
- **Regular expressions**: notations for characterising a set of strings. Useful for data pre-processing and information extraction or standardisation.

What's Next?

- Readings:
  - Jurafsky and Martin. Speech and Language Processing. Chapter 2, 2026. [recommended]
  - T. Stubblebine. Regular Expression: Pocket Reference. O'Reilly, 2007.

- Workshop **tomorrow**: Python coding for word tokenization and regular expressions.

- Next lecture: N-gram language models.